1

a. Illustration of Where Clause, AND, OR operations in MongoDB.

- **Creating database**

```
>use ProgrammingBooks
```

- **create the collection inside the database**

```
>db.createCollection("BookDetails")
```

- **Inserting Documents into collection**

Method 1: Insert the single value or document

```
>db.BookDetails.insertOne({
    _id: 1,
    title: "Clean Code",
    author: "Robert C. Martin",
    category: "Software Development",
    year: 2008
    })
```

Method 2: Insert multiples values or document in collection

```
>db.BookDetails.insertMany([
    {_id: 2, title: "Clean Code", author: "Robert C. Martin", category: "Software
        Development", year: 2008 },
    {_id: 3, title: "JavaScript: The Good Parts", author: "Douglas Crockford",
        category: "JavaScript", year: 2008 },
    {_id: 4, title: "Design Patterns", author: "Erich Gamma", category: "Software
        Design", year: 1994 },
    {_id: 5, title: "Introduction to Algorithms", author: "Thomas H. Cormen",
        category: "Algorithms", year: 2009 },
    {_id: 6, title: "Python Crash Course", author: "Eric Matthes", category:
        "Python", year: 2015 }
    ] );
```

- **Displaying documents within the collection**

```
> db.BookDetails.find( )
```

Or

```
> db.BookDetails.find( ).pretty( )
```

- **Displaying documents based on condition/s(using $where)**

```
>db.BookDetails.find({ year: 2008 }).pretty( )
```

Or

```
> db.BookDetails.find({ year:{$eq: 2008 }}).pretty( )
```

Or

```
> db.BookDetails.find({$where:"this.year==2008"}).pretty()
```

- **Combining multiple conditions using $and and $or**

Syntax: $and

```
{ $and: [ { <expression1> }, { <expression2> } , ... , { <expressionN> } ] }
```

Example:

```
db.BookDetails.find({
        $and: [
            { category: "Software Development" },
            { year: 2008 }
            ]
        }).pretty()
```

Syntax: $or

```
{ $or: [ { <expression1> }, { <expression2> } , ... , { <expressionN> } ] }
```

Example:

```
db.BookDetails.find({
        $or: [
            { category: "JavaScript" },
            { year: 2015 }
            ]
        }).pretty()
```

- **Using $in for displaying documents**

Syntax: { field: { $in: [<value1>, <value2>, ... <valueN>] } }

Example: db.BookDetails.find({category:{$in:['javascript','Software Development']}})
.pretty( )

b. Execute the Commands of MongoDB and operations in MongoDB: Insert, Query, Update, Delete and Projection. (Note: use any collection)

- **Retrieving documents based on different conditions using Comparison operators**

Name	Description
$eq	Matches values that are equal to a specified value.
$gt	Matches values that are greater than a specified value.
$gte	Matches values that are greater than or equal to a specified value.
$in	Matches any of the values specified in an array.
$lt	Matches values that are less than a specified value.
$lte	Matches values that are less than or equal to a specified value.
$ne	Matches all values that are not equal to a specified value.
$nin	Matches none of the values specified in an array.

Examples:

```
>db.BookDetails.find({ year: { $gt: 2000 } }).pretty( )
>db.BookDetails.find({ year: { $gte: 2000 } }).pretty( )
>db.BookDetails.find({ year: { $lt: 2000 } }).pretty( )
>db.BookDetails.find({ year: { $lte: 2000 } }).pretty( )
>db.BookDetails.find({ year: { $ne: 2000 } }).pretty( )
>db.BookDetails.find({category:{$in:['javascript','Software Development']}}).pretty( )
>db.BookDetails.find({category:{$nin:['javascript','Software Development']}}).pretty( )
```

Combining multiple conditions

```
>db.BookDetails.find({$and:[{year:{$gt:2010}},{category:{$eq:'Java and J2EE'}}]})
```

- **Updating the field/s of documents in the collection**
  Method 1: updating one document(using updateOne( ))

Syntax:

```
db.collection.updateOne(
        {filter},
        { $set: { field1: value1 }, $inc: { filed2: value2 } },
        ...
        )
```

Where:

Filter: The selection criteria for the update. The same **query selectors** as in the find() method are available.
Specify an empty document **{ }** to **update** the **first document** returned in the collection.

Example:

```
>db.BookDetails.updateOne(
 { title: "Clean Code: A Handbook of Agile Software Craftsmanship" },
 { $set: { author: "vtucode" } }
  )
```

Method 2: updating mutiple documents(using updateMany( ))

```
>db.BookDetails.updateMany(
 { year: { $lt: 2010 } },
 { $set: { category: "vtucode website" } }
)
```

And execute the following command to see the updations

```
> db.BookDetails.find({ year: { $lt: 2010 } }).pretty( )
```

Note: updateMany( ) can also be used to add the field to existing documents

Example: adding publisher field to all the documents

```
>db.BookDetails.updateMany({category:{$exists:true}},{$set:{pubisher:"McGrawHill"}})
```

Method 3: using findOneAndUpdate( )

Syntax:

db.collection.findOneAndUpdate(filter, update, options)

- returnDocument: **"before"** returns the original document.
- returnDocument: **"after"** returns the updated document

Example:

```
>db.BookDetails.findOneAndUpdate({category:"Python"},
{$set:{title:"Python:Complete refrence"}},{returnDocumnet:"after"})
```

Method 4: using findAndModify( )

This method can handle three types of operations:

i. **Update: Modify the existing document.**

```
>db.BookDetails.findAndModify({query:{category:'Python'},
update:{$set:{title:"Python:For Begginers",year:2015}}})

>db.BookDetails.findAndModify({query:{category:'Python
Programming'},update:{$set:{title:"Python:For Begginers",year:2015}},new:true})
```

ii. **Remove: Delete the document.**

```
>db.BookDetails.findAndModify({
  query: {category:'Python Programming'},
  sort: { score: 1 },
  remove: true
});
```

iii. **Upsert: Insert a new document if no document matches the query.**

```
>db.BookDetails.findAndModify ({query:{category:"Java and J2EE"},
update:{$set:{title:"JAVA2: complete reference",year:2012}},
upsert:true,
new:true})
```

- **Delete the documents**

  Method 1: Delete single value or document in collection

  ```
  >db.BookDetails.deleteOne({ _id: 2 })
  ```

  Method 2: Delete multiple values or document in collection

  ```
  >db.BookDetails.deleteMany({ year: { $lt: 1995 } })
  ```

  Method 3: Delete all documents

  ```
  >db.BookDetails.deleteMany({ })
  ```

- **verify the deleted document**

  ```
  >db.BookDetails.find().pretty( )
  ```

- **Deleting all documents:**

  ```
  >db.BookDetails.deleteMany({ })
  ```

  Verify the details of the collections by executing the command:

  ```
  >db.BookDetails.find().pretty( )
  ```

	➢ **Projection Operations: To retrieve specific include field values or document in collection** **By using 1 and 0 can retrieve the details of specific fields of the documents** >db.BookDetails.find({ },{category:1,title:1,price:1}) **//with _id filed by default** >db.BookDetails.find({},{category:1,title:1,price:1,_id:0}) **//exclude _id field** >db.BookDetails.find({title:'Clean Code'},{category:1,title:1,price:1,_id:0}) **//with condition**
2	**a. Develop a MongoDB query to select certain fields and ignore some fields of the documents from any collection.** **To retrieve specific include field values or document in collection** **By using 1 and 0 can retrieve the details of specific fields of the documents** ➢ **Including Specific Fields:** >db.BookDetails.find({ },{category:1,title:1,price:1}) **//with _id filed by default** >db.BookDetails.find({},{category:1,title:1,price:1,_id:0}) **//exclude _id field** ➢ **Combining Filter and Projection:** >db.BookDetails.find({title:'Clean Code'},{category:1,title:1,price:1,_id:0}) **//with condition** ➢ **Excluding Specific Fields:** >db.BookDetails.find({title:'Clean Code'},{category:0,_id:0}) **//excluding multiple fields** **b. Develop a MongoDB query to display the first 5 documents from the results obtained.[use of limit and find]** >db.BookDetails.countDocuments( ) **//displays number of documents in the collection** >db.BookDetails.find().limit(5) **//displays first 5 documents** >db.BookDetails.find({},{category:1,title:1,price:1,_id:0}).limit(5) >db.BookDetails.find().skip(2) **//displays 3rd document onwards by skipping first 2** **Sorting the documents using sort( )** >db.BookDetails.find().skip(2).sort('category') >db.BookDetails.find().sort().limit(4) >db.BookDetails.find({},{title:1,author:1,year:1,_id:0}).sort().limit(4) >db.BookDetails.find({},{title:1,author:1,year:1,_id:0}).sort("year").limit(6)

```
>db.BookDetails.find({},{title:1,author:1,year:1,_id:0}).sort({"year":-1}).limit(6)
>db.BookDetails.find({},{title:1,author:1,year:1,_id:0}).sort("created_at",-1).limit(4)
```

3 **Execute query selectors (comparison selectors, logical selectors) and list out the results on any collection**

- **create the new database as well as switch the database**

```
>use companyDB
```

- **insert the multiple values or documents**

```
>db.Employees.insertMany([
    { name: "Alice", age: 30, department: "HR", salary: 50000,
        joinDate: new Date("2015-01-15") },
    { name: "Bob", age: 24, department: "Engineering", salary: 70000,
        joinDate: new Date("2019-03-10") },
    { name: "Charlie", age: 29, department: "Engineering", salary: 75000,
        joinDate: new Date("2017-06-23") },
    { name: "David", age: 35, department: "Marketing", salary: 60000,
        joinDate: new Date("2014-11-01") },
    { name: "Eve", age: 28, department: "Finance", salary: 80000,
        joinDate: new Date("2018-08-19") }
])
```

- **comparison selectors:**

Operator	Description	Example
$eq	Equal to	{ age: { $eq: 25 } }
$ne	Not equal to	{ status: { $ne: "A" } }
$gt	Greater than	{ age: { $gt: 30 } }
$gte	Greater than or equal to	{ age: { $gte: 18 } }
$lt	Less than	{ age: { $lt: 60 } }
$lte	Less than or equal to	{ age: { $lte: 40 } }
$in	Value in an array	{ status: { $in: ["A", "B"] } }
$nin	Value not in an array	{ status: { $nin: ["C"] } }

- **$eq (Equal)**

  >db.Employees.find({ department: { $eq: "Engineering" } }).pretty( )

- **$ne (Not Equal)**

  >db.Employees.find({ department: { $ne: "HR" } }).pretty( )

- **$gt (Greater than)**

  >db.Employees.find({ age: { $gt: 30 } }).pretty()

- **$lt (less than)**

  >db.Employees.find({ salary: { $lt: 70000 } }).pretty()

- **$gte (Greater Than or Equal)**

  >db.Employees.find({ joinDate: { $gte: new Date("2018-01-01") } }).pretty()

- **$lte (Less Than or Equal)**

  >db.Employees.find({ age: { $lte: 28 } }).pretty()

- **logical selectors**

Operator	Description	Example Use
$and	Matches documents that match all conditions	All must be true
$or	Matches documents that match any condition	At least one must be true
$not	Inverts the result of a condition	Opposite of the condition
$nor	Matches documents that fail all conditions	None should be true

- **$and (Logical AND)**

  Example: Find employees who are in the "Engineering" department and have a salary greater than 70000.

  >db.Employees.find({ $and: [{ department: "Engineering" }, { salary: { $gt: 70000 } }] }).pretty( )

- **$or (Logical OR)**

  Example: Find employees who are either in the "HR" department or have a salary less than 60000.

  >db.Employees.find({ $or: [{ department: "HR" }, { salary: { $lt: 60000 } }] }).pretty()

- **$not (Logical NOT)**

  Example: Find employees who are not in the "Engineering" department.

```
>db.Employees.find({ department: { $not: { $eq: "Engineering" } } }).pretty( )
```

- **$nor (Logical NOR)**

Example: Find employees who are neither in the "HR" department nor have a salary greater than 75000.

```
>db.Employees.find({ $nor: [ { department: "HR" }, { salary: { $gt: 75000 } } ] }).pretty()
```

- **Execute query selectors (Geospatial selectors, Bitwise selectors) and list out the results on any collection**

- **Geospatial selectors:**
- Geospatial selectors in MongoDB are used to store and query geospatial data like coordinates, areas, and distances.
- MongoDB supports 2D (flat plane) and 2dsphere (Earth-like) indexes, and has powerful query operators for geospatial operations.
- To specify **GeoJSON** data, use an **embedded document** with:

  a field named **type** that specifies the GeoJSON object type, and
  a field named **coordinates** that specifies the object's coordinates.
  <field>: { type: <GeoJSON type> , coordinates: <coordinates> }

- **Create collection**

```
>use geodatabase
```

- **Insert the documents describing the places**

```
>db.Places.insertMany([
 { name: "Central Park", location: { type: "Point", coordinates: [-73.9654, 40.7829] } },
 { name: "Times Square", location: { type: "Point", coordinates: [-73.9851, 40.7580] } },
 { name: "Brooklyn Bridge", location: { type: "Point", coordinates: [-73.9969, 40.7061] } },
 { name: "Empire State Building", location: { type: "Point", coordinates: [-73.9857, 40.7488] } },
 { name: "Statue of Liberty", location: { type: "Point", coordinates: [-74.0445, 40.6892] } }
])
```

Note:

- Geospatial indexes in MongoDB are designed to efficiently perform spatial queries, like finding locations near a specific point, within a given radius, or within a particular shape.
- They allow MongoDB to quickly filter and return documents based on their geographical location without scanning the entire collection.

- **Create a geospatial index**

```
db.Places.createIndex({ location: "2dsphere" })
```

- **$near (Find places near a certain point):**

  Example: Find places near a specific coordinate, for example, near Times Square.

```
db.Places.find({
 location: {
  $near: {
   $geometry: {
    type: "Point",
    coordinates: [-73.9851, 40.7580]
   },
   $maxDistance: 5000                    // distance in meters
  }
 }
}).pretty( )
```

- **$geoWithin (Find places within a specific area):**

  Example: Find places within a specific polygon, for example, an area covering part of Manhattan.

```
db.Places.find({
 location: {
  $geoWithin: {
   $geometry: {
    type: "Polygon",
    coordinates: [
     [
      [-70.016, 35.715],
      [-74.014, 40.717],
      [-73.990, 40.730],
      [-73.990, 40.715],
```

```
            [-70.016, 35.715]
          ]
        ]
      }
    }
  }
}).pretty()
```

➢ Bitwise selectors

- **Bitwise selectors** are query operators that allow you to perform **bit-level queries** on **integer fields**.
- These selectors are useful when you store multiple Boolean flags or states in a single integer field using **bit masking**.

Bit Position	Meaning	Binary	Decimal
0	Is Active	0001	1
1	Is Manager	0010	2
2	Has Remote Access	0100	4
3	Is On Probation	1000	8

Example:
An employee with flags = 5 (binary 0101) means:

- Is Active ✅
- Is Manager ❌
- Has Remote Access ✅
- Is On Probation ❌

➢ Create database

```
>use empDB
```

➢ Create collection

```
>db.createCollection("employees")
```

➢ Insert documents into the collection

```
>db.employees.insertMany([
 { name: "Alice", flags: 1 },                    // Active
```

```
 { name: "Bob", flags: 3 },                    // Active + Manager
 { name: "Charlie", flags: 5 },                // Active + Remote Access
 { name: "David", flags: 10 },                 // Manager + Probation
 { name: "Eve", flags: 15 }                    // All flags set
 ])
```

1. Get all active employees (bit 0 set)

```
db.employees.find({ flags: { $bitsAllSet: 1 } })
```

2. Get employees who are either managers OR have remote access (bit 1 or 2 set)

```
db.employees.find({ flags: { $bitsAnySet: [1, 2] } })
```

3. Get employees who are not on probation (bit 3 is clear)

```
db.employees.find({ flags: { $bitsAllClear: 8 } })
```

4. Get employees who are active and managers (bit 0 and 1 set)

```
db.employees.find({ flags: { $bitsAllSet: [0, 1] } })
```

5. Get employees where any flag is clear (not all permissions)

```
db.employees.find({ flags: { $bitsAnyClear: [0, 1, 2, 3] } })
```

4 **Create and demonstrate how projection operators ($, $elematch and $slice) would be used in the MongoDB.**

- **Create database**

```
>use retailDB
```

- **Create collection**

```
>db.creteCollection('Products')
```

- **Insert documents into the collection**

```
db.Products.insertMany([
 {
  name: "Laptop",
  brand: "BrandA",
  features: [
   { name: "Processor", value: "Intel i7" },
   { name: "RAM", value: "16GB" },
   { name: "Storage", value: "512GB SSD" }
```

```
  ],
  reviews: [
    { user: "Alice", rating: 5, comment: "Excellent!" },
    { user: "Bob", rating: 4, comment: "Very good" },
    { user: "Charlie", rating: 3, comment: "Average" }
  ]
},



{
  name: "Smartphone",
  brand: "BrandB",
  features: [
    { name: "Processor", value: "Snapdragon 888" },
    { name: "RAM", value: "8GB" },
    { name: "Storage", value: "256GB" }
  ],
  reviews: [
    { user: "Dave", rating: 4, comment: "Good phone" },
    { user: "Eve", rating: 2, comment: "Not satisfied" }
  ]
}
])
```

Projection Operators:

1. **The $ Projection Operator:** used to project the first matching element from an array of embedded documents.

Example: Find the product named "Laptop" and project the review from the user "Alice".

```
> db.Products.find(
    { name: "Laptop", "reviews.user": "Alice" },
    { "reviews.$": 1 }
  ).pretty( )
```

Using aggregate pipelining

```
>db.Products.aggregate([
    {$match:{$and:[{ name:{$eq:"Laptop"},"reviews.user":{$eq:"Alice"}}] } },
    {$project:{reviews:"$reviews"} }
    ]).pretty()
```

2. **The $elemMatch Projection Operator:** used to project the first matching element from an array based on specified criteria.

Example: Find the product named "Laptop" and project the review where the rating is greater than 4.

```
> db.Products.find(
    { name: "Laptop" },
    { reviews: { $elemMatch: { rating: { $gt: 4 } } } }
    ).pretty( )
```

3. **$slice Projection Operator:** used to include a subset of the array field.

Example: Find the product named "Smartphone" and project the first review.

```
>db.Products.find(
    { name: "Smartphone" },
    { reviews: { $slice: 1 } }
    ).pretty( )
```

Example with Multiple Projection Operators: Find the product named "Laptop" and project the name, the first two features, and the review with the highest rating.

```
> db.Products.find( { name: "Laptop" },
{ name: 1,
features: { $slice: 2 },
reviews: { $elemMatch: { rating: 5 } } } ).pretty( )
```

5

Execute Aggregation operations ($avg, $min,$max, $push, $addToSet etc.). students encourage to execute several queries to demonstrate various aggregation operators)

- **Create database**

```
> use salesDB
```

- **Create collection**

```
>db.createCollection('Sales')
```

```
> db.Sales.insertMany([
{ date: new Date("2024-01-01"), product: "Laptop", price: 1200, quantity: 1, customer: "Amar" },
{ date: new Date("2024-01-02"), product: "Laptop", price: 1200, quantity: 2, customer: "Babu" },
```

```
{ date: new Date("2024-01-03"), product: "Mouse", price: 25, quantity: 5, customer: "Chandra" },
{ date: new Date("2024-01-04"), product: "Keyboard", price: 45, quantity: 3, customer: "Amar" },
{ date: new Date("2024-01-05"), product: "Monitor", price: 300, quantity: 1, customer: "Babu" },
{ date: new Date("2024-01-06"), product: "Laptop", price: 1200, quantity: 1, customer: "Deva" }
])
```

Aggregation Operations:

1. $avg (Average): Calculate the average price of each product.

```
> db.Sales.aggregate([ { $group: { _id: "$product",
                                   averagePrice: { $avg: "$price" } } } ]).pretty( )
```

2. $min (Minimum): Find the minimum price of each product.

```
> db.Sales.aggregate([ { $group: { _id: "$product",
                                  minPrice: { $min: "$price" } } } ]).pretty()
```

3. $max (Maximum): Find the maximum price of each product.

```
> db.Sales.aggregate([ { $group: { _id: "$product",
                                   maxPrice: { $max: "$price" } } } ]).pretty()
```

4. $push (Push Values to an Array): Group sales by customer and push each purchased product into an array.

```
> db.Sales.aggregate([ { $group: { _id: "$customer",
                                  products: { $push: "$product" } } } ]).pretty()
```

5. $addToSet (Add Unique Values to an Array): Group sales by customer and add each unique purchased product to an array.

```
> db.Sales.aggregate([ { $group: { _id: "$customer",
                                 uniqueProducts: { $addToSet: "$product" } } } ]).pretty()
```

Combining Aggregation Operations:

Example: Calculate the total quantity and total sales amount for each product, and list all customers who purchased each product.

```
db.Sales.aggregate([
 {
  $group: {
```

```
    _id: "$product",
    totalQuantity: { $sum: "$quantity" },
    totalSales: { $sum: { $multiply: ["$price", "$quantity"] } },
    customers: { $addToSet: "$customer" }
  }
 }
]).pretty( )
```

6

Execute Aggregation Pipeline and its operations (pipeline must contain $match, $group, $sort, $project,$skip etc. students encourage to execute several queries to demonstrate various aggregation operators)

- **Create database**

```
>use restaurantDB
```

- **Create collection**

```
>db.createCollection('restaurants')
```

- **Insert documents**

```
db.restaurants.insertMany([
 {
  name: "Biryani House",
  cuisine: "Indian",
  location: "Jayanagar",
  reviews: [
   { user: "Aarav", rating: 5, comment: "Amazing biryani!" },
   { user: "Bhavana", rating: 4, comment: "Great place!" }
  ]
 },
 {
  name: "Burger Joint",
  cuisine: "American",
  location: "Koramangala",
  reviews: [
```

```
      { user: "Chirag", rating: 3, comment: "Average burger" },
      { user: "Devika", rating: 4, comment: "Good value" }
    ]
  },
  {
    name: "Pasta House",
    cuisine: "Italian",
    location: "Rajajinagar",
    reviews: [
      { user: "Esha", rating: 5, comment: "Delicious pasta!" },
      { user: "Farhan", rating: 4, comment: "Nice ambiance" }
    ]
  },


 {
    name: "Curry Palace",
    cuisine: "Indian",
    location: "Jayanagar",
    reviews: [
      { user: "Gaurav", rating: 4, comment: "Spicy and tasty!" },
      { user: "Harini", rating: 5, comment: "Best curry in town!" }
    ]
  },
  {
    name: "Taco Stand",
    cuisine: "Mexican",
    location: "Jayanagar",
    reviews: [
      { user: "Ishaan", rating: 5, comment: "Fantastic tacos!" },
      { user: "Jaya", rating: 4, comment: "Very authentic" }
    ]
  }
])
```

Aggregation Pipeline and its operations:

- **Execute Aggregation Pipeline and its operations (pipeline must contain match, group, sort, project, $skip etc.)**

```
db.restaurants.aggregate([
```

```
  {
    $match: {
      "location": "Jayanagar"
    }
  },
  {
    $unwind: "$reviews"
  },
  {
    $group: {
      _id: "$name",
      averageRating: { $avg: "$reviews.rating" },
      totalReviews: { $sum: 1 }
    }
  },
  {
    $sort: {
      averageRating: -1
    }
  },
  {
    $project: {
_id: 0,
      restaurant: "$_id",
      averageRating: 1,
      totalReviews: 1
    }
  },
  {
    $skip: 1
  }
]).pretty( )
```

Note: Execution of query

1. **$match:** Filter restaurants by cuisine ("Jayanagar" location).
2. **$unwind:** Deconstruct the reviews array from each document to output a document for each review.
3. **$group:** Group the documents by restaurant name and calculate the average rating and total number of reviews.
4. **$sort:** Sort the results by average rating in descending order.

	5. **$project:** Restructure the output to include only the restaurant name, average rating, and total reviews. 6. **$skip:** Skip the first document.

7	**a. Find all listings with listing_url, name, address, host_picture_url in the listings And Reviews collection that have a host with a picture url**
	
	➢ **Create database**
	> use vacationRentals
	➢ **Create collection**
	>db.createCollection('listingsAndReviews')
	
	➢ **Insert documents**
	
	db.listingsAndReviews.insertMany([
	{
	listing_url: "http://www.example.com/listing/123456",
	name: "Beautiful Apartment",
	address: {
	street: "123 Main Street",
	suburb: "Central",
	city: "Metropolis",
	country: "Wonderland"
	},
	host: {
	name: "Alice",
	picture_url: "http://www.example.com/images/host/host123.jpg"

```
    }
  },
  {
    listing_url: "http://www.example.com/listing/654321",
    name: "Cozy Cottage",
    address: {
      street: "456 Another St",
      suburb: "North",
      city: "Smallville",
      country: "Wonderland"
    },
    host: {
      name: "Bob",
      picture_url: ""
    }
  },


  {
    listing_url: "http://www.example.com/listing/789012",
    name: "Modern Condo",
    address: {
      street: "789 Side Road",
      suburb: "East",
      city: "Gotham",
      country: "Wonderland"
    },
    host: {
      name: "Charlie",
      picture_url: "http://www.example.com/images/host/host789.jpg"
    }
  }
])
```

- **Query to Find Listings with Host Picture URLs: (use $exists)**

```
db.listingsAndReviews.find(
  {
    "host.picture_url": { $exists: true, $ne: "" }
  },
  {
    listing_url: 1,
```

```
        name: 1,
        address: 1,
        "host.picture_url": 1
      }
    ).pretty()
```

b. Using E-commerce collection write a query to display reviews summary.

- **Create database**

```
> use ecommerce
```

- **Create collection**

```
>db.createCollection('products')
```

- **Insert documents**

```
db.products.insertMany([
 {
 product_id: 1,
 name: "Laptop",
 category: "Electronics",
 price: 1200,
 reviews: [
  { user: "Alice", rating: 5, comment: "Excellent!" },
  { user: "Bob", rating: 4, comment: "Very good" },
  { user: "Charlie", rating: 3, comment: "Average" }
 ]
},
{
 product_id: 2,
 name: "Smartphone",
 category: "Electronics",
 price: 800,
 reviews: [
  { user: "Dave", rating: 4, comment: "Good phone" },
  { user: "Eve", rating: 2, comment: "Not satisfied" },
  { user: "Frank", rating: 5, comment: "Amazing!" }
```

```
  ]
 },
{
  product_id: 3,
  name: "Headphones",
  category: "Accessories",
  price: 150,
  reviews: [
   { user: "Grace", rating: 5, comment: "Great sound" },
   { user: "Heidi", rating: 3, comment: "Okay" }
  ]
 }
])
```

- **Query to Find E-commerce collection:**

Note: To display a summary of reviews in an e-commerce collection, we can assume the ecommerce database contains a products collection with documents structured to include reviews. Each product document could have a reviews array with review details such as rating, comment, and user.

```
>db.products.aggregate([
 {
  $unwind: "$reviews"
 },
 {
  $group: {
   _id: "$name",
   totalReviews: { $sum: 1 },
   averageRating: { $avg: "$reviews.rating" },
   comments: { $push: "$reviews.comment" }
  }
 },
 {
  $project: {
```

```
      _id: 0,
      product: "$_id",
      totalReviews: 1,
      averageRating: 1,
  comments: 1
    }
  }
]).pretty( )
```

Explanation:

1. **$unwind:** Deconstructs the reviews array from each document to output a document for each element.
2. **$group:** Groups the documents by product name, and calculates:
   - totalReviews: The total number of reviews for each product.
   - averageRating: The average rating of the reviews for each product.
   - comments: An array of all review comments for each product.
3. **$project:** Restructures the output documents to include the product name, total reviews, average rating, and comments.

8

a. Demonstrate creation of different types of indexes on collection (unique, sparse, compound and multikey indexes)

➢ **Insert Sample Documents**

```
db.employees.insertMany([
  { name: "Alice", email: "alice@example.com", dept: "HR", skills: ["communication", "leadership"]
  },
{ name: "Bob", email: "bob@example.com", dept: "Engineering", skills: ["JavaScript", "MongoDB"]
  },
{ name: "Charlie", dept: "HR", skills: ["recruitment"] },
{ name: "David", email: "david@example.com", dept: "Engineering" },
{ name: "Eve", email: "eve@example.com", dept: "Finance", skills: ["accounting", "Excel"] }])
```

1. Unique Index on Email

```
>db.employees.createIndex({ email: 1 }, { unique: true })
```

- Ensures no two employees can have the same email.
- If you try to insert another document with an existing email, MongoDB will throw a duplicate key error.

2. Sparse Index on Email

```
>db.employees.createIndex({ email: 1 }, { sparse: true })
```

- This only indexes documents where email exists, so it allows multiple documents without an email field without violating the unique constraint (if unique: true is combined).

3. Compound Index on Department and Name

```
>db.employees.createIndex({ dept: 1, name: 1 })
```

Note: Optimizes queries like:

```
db.employees.find({ dept: "HR", name: "Alice" })
```

4. Multikey Index on Skills (Array Field)

```
>db.employees.createIndex({ skills: 1 })
```

Note: This creates a multikey index because skills is an array. Enables fast queries like:

```
db.employees.find({ skills: "MongoDB" })
```

5. View All Indexes on the Collection

```
>db.employees.getIndexes( )
```

b.Demonstrate optimization of queries using indexes.

Insert Sample Documents

```
>db.products.insertMany([
 { name: "Laptop", category: "Electronics", price: 1000, stock: 10 },
  { name: "Smartphone", category: "Electronics", price: 500, stock: 50 },
 { name: "Desk Chair", category: "Furniture", price: 150, stock: 5 },
{ name: "Notebook", category: "Stationery", price: 5, stock: 100 },
{ name: "Monitor", category: "Electronics", price: 200, stock: 20 }
])
```

Run a Query Without an Index

```
>db.products.find({ category: "Electronics", price: { $lt: 600 } }).explain("executionStats")
```

Create an Index on category and price

```
>db.products.createIndex({ category: 1, price: 1 })
```

	➢ **Re-run the Same Query with explain( ) as follows** >db.products.find({ category: "Electronics", price: { $lt: 600 } }).explain("executionStats").executionStats. totalDocsExamined >db.products.find({ category: "Electronics", price: { $lt: 600 } }).explain("executionStats").executionStats. executionTimeMillisEstimate **Note:** check the following fields in the output : totalDocsExamined should be lower executionTimeMillis should be reduced indexName should match the index used
9	**a. Develop a query to demonstrate Text search using catalog data collection for a given word** ➢ **Create database** > use kannadaMovies ➢ **Create collection** >db.createCollection('catalog') ➢ Add documents through the **csv file** consisting of multiple documents(using Adddata button in mongoDB compass) **Verify the documents loaded for the collection** >db.catalog.countDocuments( ) **// number of documents** ➢ **Create a Text Index** Next, create a text index on the **name** and **genre** fields to enable text search. >db.catalog.createIndex({name: "text", genre: "text"})

- **Perform a Text Search Query**

Now, let's perform a text search to find documents containing a **specific word**.

```
>db.catalog.find({$text: {$search: "maga"}})        //for name field
>db.catalog.find({$text: {$search: "comedy"}})      // for genre field
```

- **Perform a Text Search Query for a phrase**

```
>db.catalog.find({$text: {$search: ""tappida Maga""}})
```

b.Develop queries to illustrate excluding documents with certain words and phrases

- **Perform Queries to Exclude Documents**

Use the **$text** operator **with negation** to exclude documents containing specific words or phrases.

Example 1: Exclude Documents Containing the Word "action"

```
>db.catalog.find({ $text: { $search: "crime romance -action" }, year:2021 } )
```

Example 2: Exclude Documents Containing the Phrase "da maga"

```
> db.catalog.find({$text: {$search: 'maga  -"da maga" '}})
```

or

```
>db.catalog.find({ $text: { $search: "maga -\\\"da maga\\\"" } })
```

10 **Develop an aggregation pipeline to illustrate Text search on Catalog data collection.**

- The aggregation framework in MongoDB is a powerful tool for data processing and transformation.
- It consists of a series of stages, each stage performing an operation on the input documents and passing the results to the next stage. This sequence of operations is called an **aggregation pipeline.**
- Step-by-Step guide to using the aggregation pipeline in MongoDB, with examples of various stages such as **$match, $group, $sort, $project, $skip,** and **others**.

Example: Demonstrating projection stage

```
>db.catalog.find({$project:{year:'$year'}})

>db.catalog.aggregate({$project:{year:'$year',genre:1}})
```

Or

```
>db.catalog.aggregate({$project:{year:'$year',genre:'$genre'}})
```

Example: Demonstrating projection and limit stage

```
>db.catalog.aggregate({$project:{year:'$year',genre:'$genre'}},{$limit:5})
```

Example : create an aggregation pipeline that includes various stages:

$match: Filter documents to include only those in the year 2017.
$group: Group documents by genre and calculate the average rating for each genre.
$sort: Sort the results by avgRating in descending order.
$project: Include specific fields in the output
$limit: Limit the output to 5 results.
$skip: Skip the first two results

```
result = db.catalog.aggregate([
{$match: {year :2017}},                          // 1. Match stage: Filter documents by year 2017
{$group: {_id: "$genre", avgRating:{$avg: "$rating"}}}, // 2. Group stage: Group by genre and
                                                          calculate average rating
{$sort: {avgRating:-1}},                         // 3. Sort stage: Sort by avgRating in descending order
{$project: {year:"$year", avgRating:1, genre:1} },     // 4. Project stage: Include specific fields
{$limit:5} ]).toArray()                          // 5. Limit stage: Limit the output to 5 results
print("Top 5 rated movie genres with their average rating")
printjson(result)
```

Adding another stage using '$skip':

Example 2: Find the remaining five genres among the top-rated seven genres after skipping the first two, we can use the $skip stage in the aggregation pipeline.

```
result2 = db.catalog.aggregate([
  {$match: {year :2017}},                        // 1. Match stage: Filter documents by year 2017
{$group: {_id: "$genre", avgRating:{$avg: "$rating"}}}, // 2. Group stage: Group by genre and
                                                          calculate average rating
{$sort: {avgRating:-1}},                         // 3. Sort stage: Sort by avgRating in descending order
{$project: {year:"$year", avgRating:1, genre:1} },     // 4. Project stage: Include specific fields

  {$limit:7},                                    // 5. Limit stage: Limit the output to 7 results
{$skip:2} ]).toArray( )                          // 6. Skip stage: Skip the first two results
print("Top 7 rated movie genres with their average rating with the first two skipped")
printjson(result2)
```

```
db.Products.aggregate([
 {
  $match: { name: "Laptop" }
 },
 {
  $project: {
   reviews: {
    $filter: {
     input: "$reviews",
     as: "review",
     cond: { $eq: [ "$$review.user", "Alice" ] }
    }
   }
  }
 }
]).pretty()
```